

Trabajo práctico final

Aplicaciones móviles

Alumno: Gauna, Luciano Ezequiel

Docente: Medina, Sergio Daniel

Comisión: ACN4BV

Introducción

El presente informe describe el desarrollo realizado para el trabajo final de la materia. El proyecto consiste en la ampliación de una aplicación Android nativa orientada a la compra de productos de supermercado, tomando como base el proyecto desarrollado en instancias anteriores.

En esta etapa final se incorporó Firebase Firestore para guardar, consultar y actualizar información dentro de la aplicación. A partir de esta integración, se agregó la creación y edición de datos del perfil del usuario, la carga de productos desde una colección remota y la generación dinámica de la pantalla de bebidas mediante RecyclerView.

Además, se reemplazó el comportamiento anterior basado en productos hardcodeados por un flujo más dinámico: los productos se obtienen desde Firestore, se muestran en tarjetas reutilizables, pueden abrirse en una pantalla de detalle y luego agregarse a un carrito dinámico. El carrito permite visualizar los productos seleccionados, modificar cantidades, eliminar elementos y calcular el total estimado del pedido.

El proyecto fue desarrollado en Android Studio utilizando Java y XML para la construcción de las interfaces. Se trabajó con recursos organizados en archivos de strings, colors, drawables e imágenes locales, buscando mantener una experiencia de usuario coherente en todas las pantallas y respetar el diseño visual definido para la aplicación.

Alcance de la implementación

Para esta entrega se desarrolló un flujo visual y funcional entre las principales pantallas de la aplicación. La pantalla de exploración permite acceder a la categoría Bebidas, desde donde se visualiza un listado dinámico de productos obtenidos desde Firebase Firestore. Cada producto puede abrirse en una pantalla de detalle, donde se muestra su información completa y se permite modificar la cantidad, marcarlo como favorito y agregarlo al carrito.

A diferencia de etapas anteriores, en esta versión se reemplazó la implementación estática de productos por una lógica dinámica basada en clases y listas reutilizables. La pantalla de bebidas ya no depende de tarjetas hardcodeadas en XML, sino que utiliza un RecyclerView y un Adapter para generar una tarjeta por cada producto obtenido desde la colección products de Firestore.

También se actualizó la pantalla de carrito para que refleje los productos agregados por el usuario desde la pantalla de bebidas o desde el detalle de producto. El carrito permite visualizar los productos seleccionados, aumentar o disminuir cantidades, eliminar elementos y calcular el total estimado del pedido. Para esto se incorporaron clases específicas como CartItem, CartManager y CartAdapter, encargadas de representar, administrar y mostrar dinámicamente los elementos del carrito.

Además, se incorporó Firestore para gestionar información del usuario. Al registrar una cuenta se crea un documento asociado al usuario autenticado, y desde la pantalla Cuenta se pueden consultar y actualizar datos como nombre, teléfono y dirección. La autenticación mediante Firebase Authentication se mantiene como base del proyecto, permitiendo login, registro, persistencia de sesión y cierre de sesión.

El alcance final de esta implementación se centra en integrar autenticación, base de datos remota, navegación entre pantallas, pasaje de datos entre Activities, renderizado dinámico de productos y comportamiento interactivo del carrito dentro de una aplicación Android nativa desarrollada con Java y XML.

Objetivo del trabajo

El objetivo principal del trabajo fue ampliar la aplicación desarrollada en las entregas anteriores, incorporando funcionalidades que permitan construir un flujo de uso más completo, dinámico y conectado con una base de datos remota. Para ello se mantuvo la base existente del proyecto, incluyendo la autenticación mediante Firebase Authentication, y se agregaron nuevas funcionalidades relacionadas con Firebase Firestore, visualización dinámica de productos y carrito de compras.

Como objetivos específicos se plantearon los siguientes puntos:

- Incorporar Firestore para guardar, consultar y actualizar información de usuarios y productos.
- Crear y actualizar datos de perfil del usuario desde la pantalla Cuenta.
- Cargar productos desde una colección products en Firestore.
- Reemplazar las tarjetas fijas de la pantalla Bebidas por una lista dinámica implementada con RecyclerView y Adapter.
- Representar los productos mediante una clase Product.
- Mantener el pasaje de datos entre Activities para mostrar el detalle del producto seleccionado.
- Permitir agregar productos al carrito desde Bebidas y desde Detalle.
- Implementar un carrito dinámico mediante CartItem, CartManager y CartAdapter.
- Permitir modificar cantidades, eliminar productos y calcular el total estimado del pedido.
- Mantener una navegación coherente entre Login, Exploración, Bebidas, Detalle, Carrito y Cuenta.
- Organizar recursos visuales en strings, colors, drawables e imágenes locales.

Funcionalidades implementadas

Durante esta etapa final del proyecto se incorporaron distintas funcionalidades con el objetivo de ampliar la aplicación desarrollada en entregas anteriores y construir un recorrido más completo, dinámico y conectado con una base de datos remota. Las principales funcionalidades implementadas y mantenidas fueron:

Autenticación de usuarios

La aplicación mantiene el sistema de autenticación de usuarios mediante Firebase Authentication, utilizando el proveedor de correo electrónico y contraseña. Desde la pantalla de inicio de sesión, el usuario puede ingresar con una cuenta existente o registrarse con un nuevo correo y contraseña válida.

También se mantiene la persistencia de sesión, por lo que si el usuario ya inició sesión previamente, la aplicación lo redirige automáticamente a la pantalla principal de exploración sin volver a pedir sus credenciales. Desde la pantalla Cuenta, además, se permite cerrar sesión y regresar a la pantalla de login.

Gestión de perfil con Firestore

Para esta entrega se incorporó Firebase Firestore como base de datos remota. Al registrar un nuevo usuario, la aplicación crea automáticamente un documento en la colección users, utilizando como identificador el uid del usuario autenticado.

En ese documento se almacenan datos asociados al perfil, como correo electrónico, nombre, teléfono, dirección y fecha de creación. Desde la pantalla Cuenta, el usuario puede consultar su correo y editar los datos de perfil. Al presionar el botón de guardar, la información se actualiza en Firestore.

Exploración de categorías

Se mantiene una pantalla de exploración de productos donde se muestran distintas categorías de supermercado, como Frutas y verduras, Almacén, Carnes y pescados, Panadería y snacks, Lácteos y huevos y Bebidas.

La pantalla incluye una barra de búsqueda que permite filtrar las categorías visibles según el texto ingresado por el usuario. De esta forma, si el usuario escribe parte del nombre de una categoría, la interfaz actualiza dinámicamente los resultados mostrados.

En esta implementación, la categoría funcional principal es Bebidas. Las demás categorías se dejaron visibles para respetar el diseño general de la aplicación, pero muestran un mensaje indicando que estarán disponibles próximamente.

Listado dinámico de productos de bebidas

Al seleccionar la categoría Bebidas, la aplicación navega hacia una pantalla con un listado de productos pertenecientes a esa categoría. A diferencia de la implementación anterior, los productos ya no se encuentran cargados de forma fija en el XML, sino que se obtienen desde la colección products de Firebase Firestore.

Cada documento obtenido desde Firestore se convierte en un objeto Product, que contiene datos como nombre, presentación, precio, descripción, categoría e imagen asociada. Luego, la pantalla utiliza un RecyclerView junto con un ProductAdapter para generar dinámicamente una tarjeta por cada producto disponible.

Cada tarjeta muestra imagen, nombre, presentación, precio y un botón de agregado representado con el signo “+”. Al presionar este botón, el producto se agrega al carrito mediante la clase CartManager.

Detalle dinámico de producto

Se mantiene una pantalla de detalle de producto reutilizable. Esta pantalla no está fija para un único producto, sino que recibe la información desde la pantalla de bebidas mediante Intent y extras.

Los datos enviados incluyen el identificador del producto, nombre, presentación, precio, descripción, imagen e identificador del recurso visual. Con esta información, la pantalla de detalle se completa dinámicamente según el producto seleccionado.

Dentro del detalle se agregó control de cantidad, actualización del precio según la cantidad seleccionada, opción visual de favorito, descripción del producto, información nutricional, opiniones y botón de agregar al carrito. En esta versión, el botón de agregar al carrito incorpora el producto real al carrito dinámico, respetando la cantidad seleccionada por el usuario.

Carrito de compras dinámico

Para esta etapa se reemplazó la implementación estática del carrito por una versión dinámica. El carrito ya no depende de productos fijos definidos en el XML, sino que muestra los productos agregados por el usuario desde la pantalla de Bebidas o desde la pantalla de Detalle.

Para administrar esta funcionalidad se incorporaron las clases CartItem, CartManager y CartAdapter. CartItem representa un producto dentro del carrito junto con su cantidad. CartManager centraliza la lista de productos agregados y permite aumentar cantidades, disminuirlas, eliminar productos y calcular el total. CartAdapter se encarga de mostrar visualmente cada producto del carrito utilizando una fila reutilizable.

La pantalla de carrito permite visualizar los productos seleccionados, modificar cantidades, eliminar elementos y calcular el total estimado del pedido mediante el botón de checkout.

Cuenta de usuario

La aplicación cuenta con una pantalla de usuario conectada con Firebase Authentication y Firebase Firestore. En esta pantalla se muestra el correo electrónico del usuario autenticado y se cargan datos adicionales del perfil, como nombre, teléfono y dirección.

Estos datos se obtienen desde el documento correspondiente al usuario dentro de la colección users de Firestore. Además, la pantalla permite editar esa información y guardarla nuevamente en la base de datos mediante el botón de guardar cambios.

También se incluye un botón para cerrar sesión. Al cerrar sesión, Firebase elimina la sesión activa del usuario y la aplicación vuelve a la pantalla de inicio de sesión. Esto permite completar el ciclo de autenticación: registro, inicio de sesión, sesión persistente, consulta de datos de perfil, edición de información y cierre de sesión.

Navegación inferior

Se mantiene una barra de navegación inferior para acceder a las principales secciones de la aplicación. Desde ella se puede navegar entre Explorar, Carrito y Cuenta, permitiendo recorrer el flujo principal de la app.

Las opciones Tienda y Favoritos se dejaron visibles como parte del diseño general y muestran un mensaje indicando que estarán disponibles próximamente. De esta forma se respeta la estructura visual planteada para la aplicación, aunque no todas las secciones se encuentren completamente implementadas en esta etapa.

Descripción de pantallas

Pantalla de inicio de sesión

La primera pantalla de la aplicación corresponde al inicio de sesión del usuario. Desde esta pantalla se pueden realizar dos acciones principales: ingresar con una cuenta existente o registrar una nueva cuenta mediante correo electrónico y contraseña.

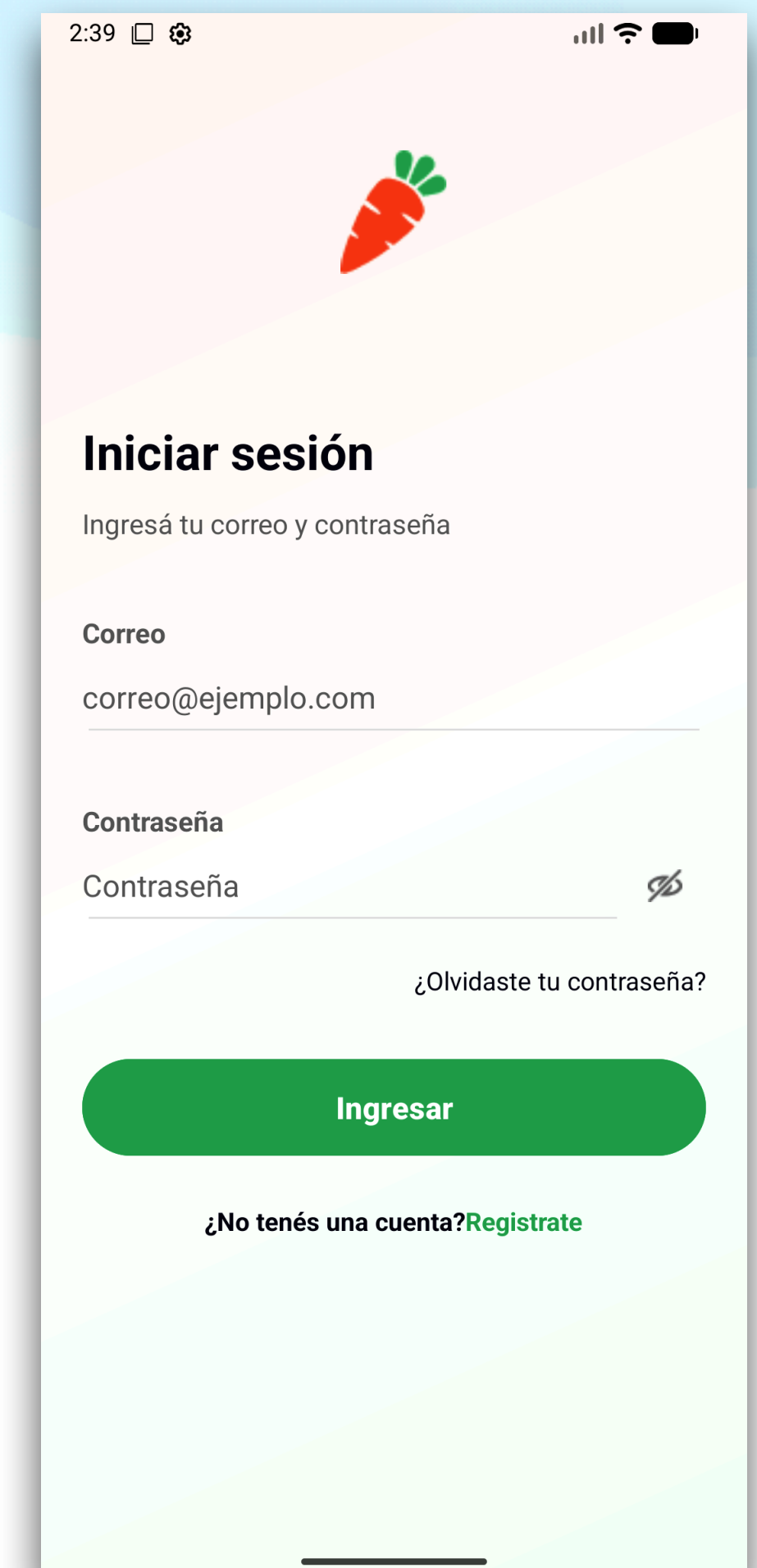
La autenticación se encuentra integrada con Firebase Authentication utilizando el proveedor de correo electrónico y contraseña. Al presionar el botón de ingreso, la aplicación valida que los campos no se encuentren vacíos y luego consulta Firebase para verificar las credenciales ingresadas. Si los datos son incorrectos o el usuario no se encuentra registrado, se muestra un mensaje informativo mediante Toast.

Desde la misma pantalla también se permite registrar un nuevo usuario. Cuando el registro se realiza correctamente, Firebase Authentication crea la cuenta y la aplicación genera además un documento asociado al usuario dentro de la colección users de Firebase Firestore. En este documento se guardan datos iniciales como correo electrónico, nombre, teléfono, dirección y fecha de creación.

En caso de intentar registrar un correo ya existente, Firebase detecta que la cuenta ya fue creada previamente y la aplicación informa al usuario mediante un mensaje. Si el inicio de sesión es correcto, la aplicación navega hacia la pantalla principal de exploración de productos.

También se implementó persistencia de sesión, por lo que si el usuario ya se encuentra autenticado, al volver a abrir la aplicación es redirigido automáticamente a la pantalla de exploración sin solicitar nuevamente sus credenciales.

La opción “¿Olvidaste tu contraseña?” se dejó implementada como acción visual dentro de la pantalla. Al presionarla se muestra un mensaje informativo mediante Toast, indicando que la recuperación de contraseña estará disponible próximamente.



2:39

Iniciar sesión

Ingresá tu correo y contraseña

Correo

correo@ejemplo.com

Contraseña

Contraseña

¿Olvidaste tu contraseña?

Ingresar

¿No tenés una cuenta? [Registrate](#)

Pantalla de exploración de productos

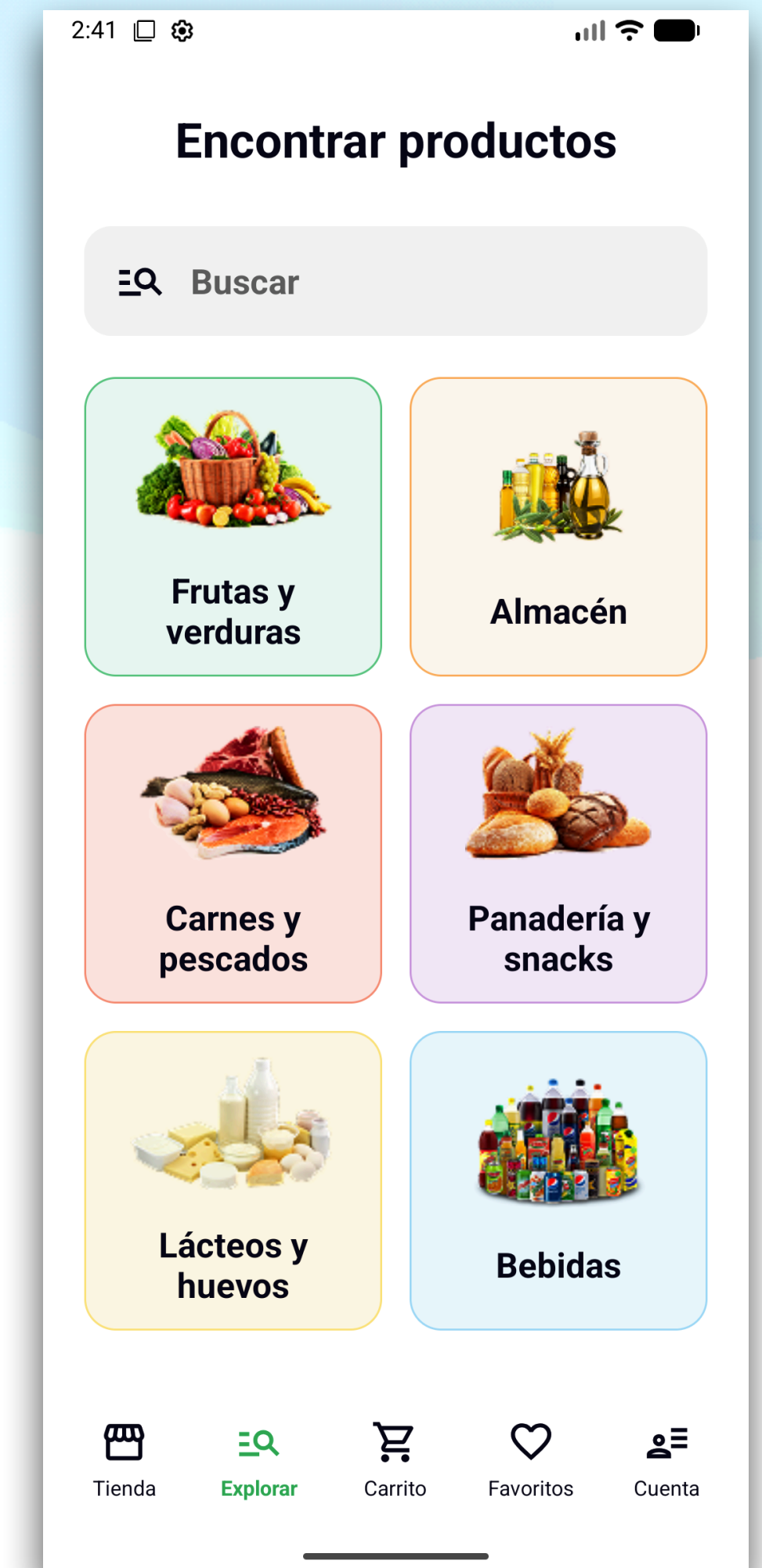
Luego de iniciar sesión correctamente, la aplicación dirige al usuario a la pantalla **Encontrar productos**. Esta pantalla funciona como punto principal de exploración del catálogo y muestra distintas categorías de supermercado organizadas en tarjetas.

Las categorías visibles son: Frutas y verduras, Almacén, Carnes y pescados, Panadería y snacks, Lácteos y huevos y Bebidas. Cada tarjeta cuenta con una imagen representativa, un color de fondo y un borde propio, respetando la estética planteada en los wireframes y manteniendo una organización visual clara para el usuario.

En esta implementación, la categoría funcional principal es **Bebidas**. Al seleccionarla, la aplicación navega hacia la pantalla correspondiente, donde se muestra el listado dinámico de productos obtenidos desde Firebase Firestore. Las demás categorías se dejaron visibles para respetar el diseño general de la aplicación, pero al seleccionarlás muestran un mensaje informativo indicando que estarán disponibles próximamente.

La pantalla también incluye un buscador funcional. Al escribir texto en la barra de búsqueda, las categorías se filtran dinámicamente según su nombre. Por ejemplo, si el usuario escribe “beb”, permanece visible la categoría Bebidas. Si no se encuentran coincidencias, se muestra un mensaje indicando que no se encontraron categorías.

En la parte inferior se incorporó una barra de navegación con las secciones Tienda, Explorar, Carrito, Favoritos y Cuenta. En esta pantalla se marca como activa la opción Explorar. Desde esta barra se puede acceder al carrito y a la cuenta del usuario, mientras que las opciones Tienda y Favoritos muestran mensajes informativos al no estar implementadas completamente en esta etapa.



Pantalla de productos - Bebidas

Al seleccionar la categoría **Bebidas** desde la pantalla de exploración, la aplicación navega hacia una nueva pantalla donde se muestra el listado de productos disponibles para esa categoría.

En esta versión, los productos no se encuentran definidos de forma fija en el archivo XML, sino que se obtienen desde Firebase Firestore. La aplicación consulta la colección products, filtra los documentos correspondientes a la categoría bebidas y convierte cada documento en un objeto Product.

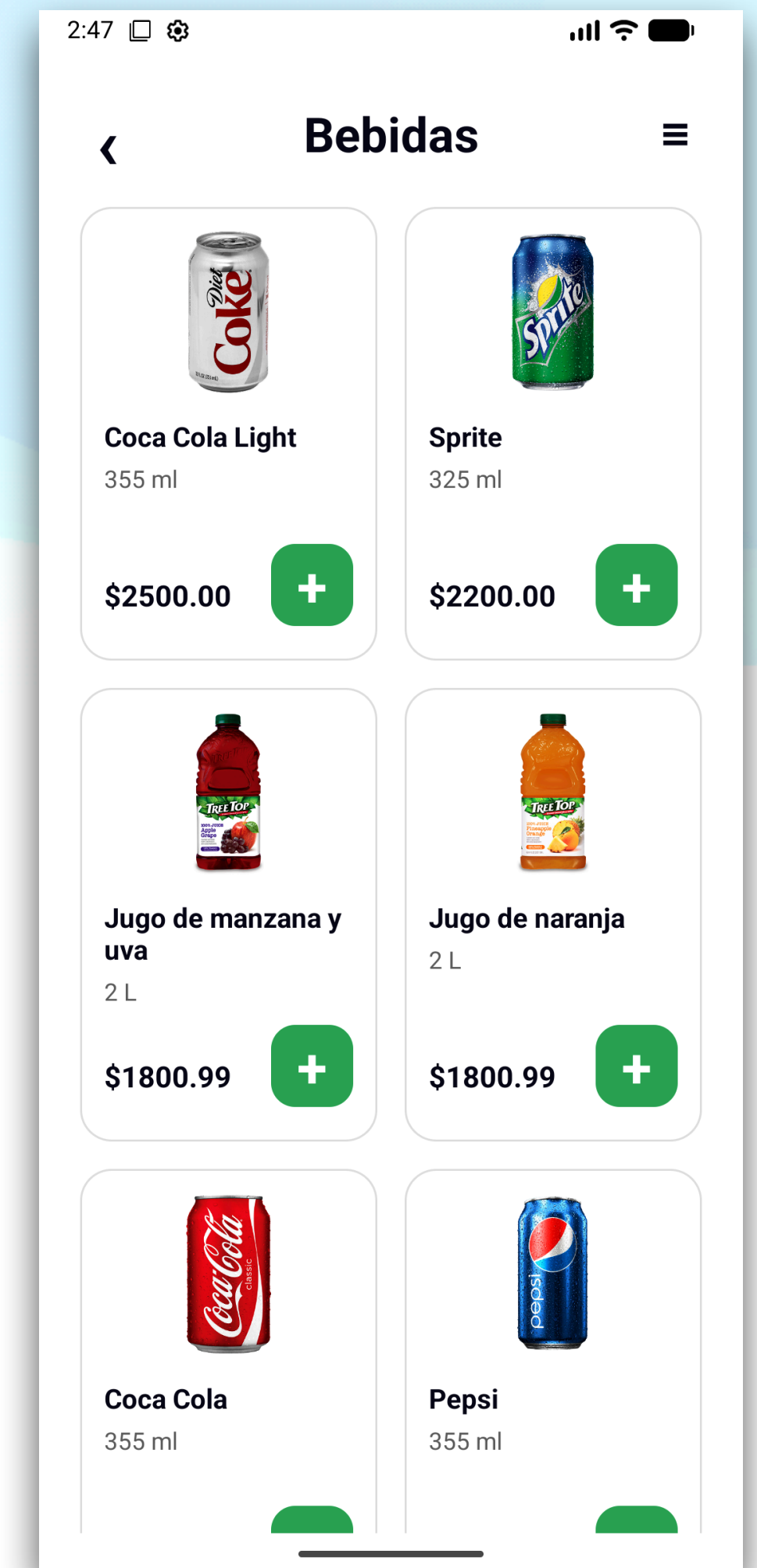
Para mostrar los productos en pantalla se utiliza un RecyclerView junto con un ProductAdapter. De esta forma, la interfaz genera dinámicamente una tarjeta por cada producto obtenido desde Firestore. Cada tarjeta utiliza un diseño reutilizable definido en item_product.xml e incluye imagen, nombre, presentación, precio y un botón de agregado.

Entre los productos cargados en Firestore se encuentran Coca Cola Light, Sprite, jugos, Coca Cola y Pepsi. Cada producto posee campos como nombre, presentación, precio, descripción, categoría e imagen asociada. En esta implementación, las imágenes se mantienen como recursos locales dentro del proyecto y Firestore almacena el nombre del recurso correspondiente.

La pantalla incluye una flecha de regreso en la parte superior izquierda, que permite volver a la pantalla de exploración. También se agregó un ícono de filtro en la parte superior derecha como elemento visual, respetando el diseño de referencia. En esta instancia, el filtro se dejó como funcionalidad futura.

Cada producto posee un botón con el signo +, que permite agregarlo directamente al carrito mediante la clase CartManager. Además, al tocar la tarjeta de un producto, la aplicación navega hacia la pantalla de detalle, enviando la información correspondiente mediante Intent y extras.

Esta pantalla funciona como paso intermedio entre la exploración por categorías y el detalle individual de cada producto, y además representa una de las principales mejoras del trabajo final al reemplazar una estructura hardcodeada por un listado dinámico conectado con Firestore.



Pantalla de detalle de producto

Al seleccionar un producto desde la pantalla de Bebidas, la aplicación navega hacia una pantalla de detalle. Esta pantalla fue implementada de forma dinámica, ya que no muestra información fija de un único producto, sino que recibe los datos del producto seleccionado desde la pantalla anterior.

Para realizar esta comunicación se utilizaron Intent y extras. Desde la pantalla de Bebidas se envían datos como el identificador del producto, nombre, presentación, precio, descripción, imagen e identificador del recurso visual. Luego, la pantalla de detalle recibe esa información y la muestra en la interfaz correspondiente.

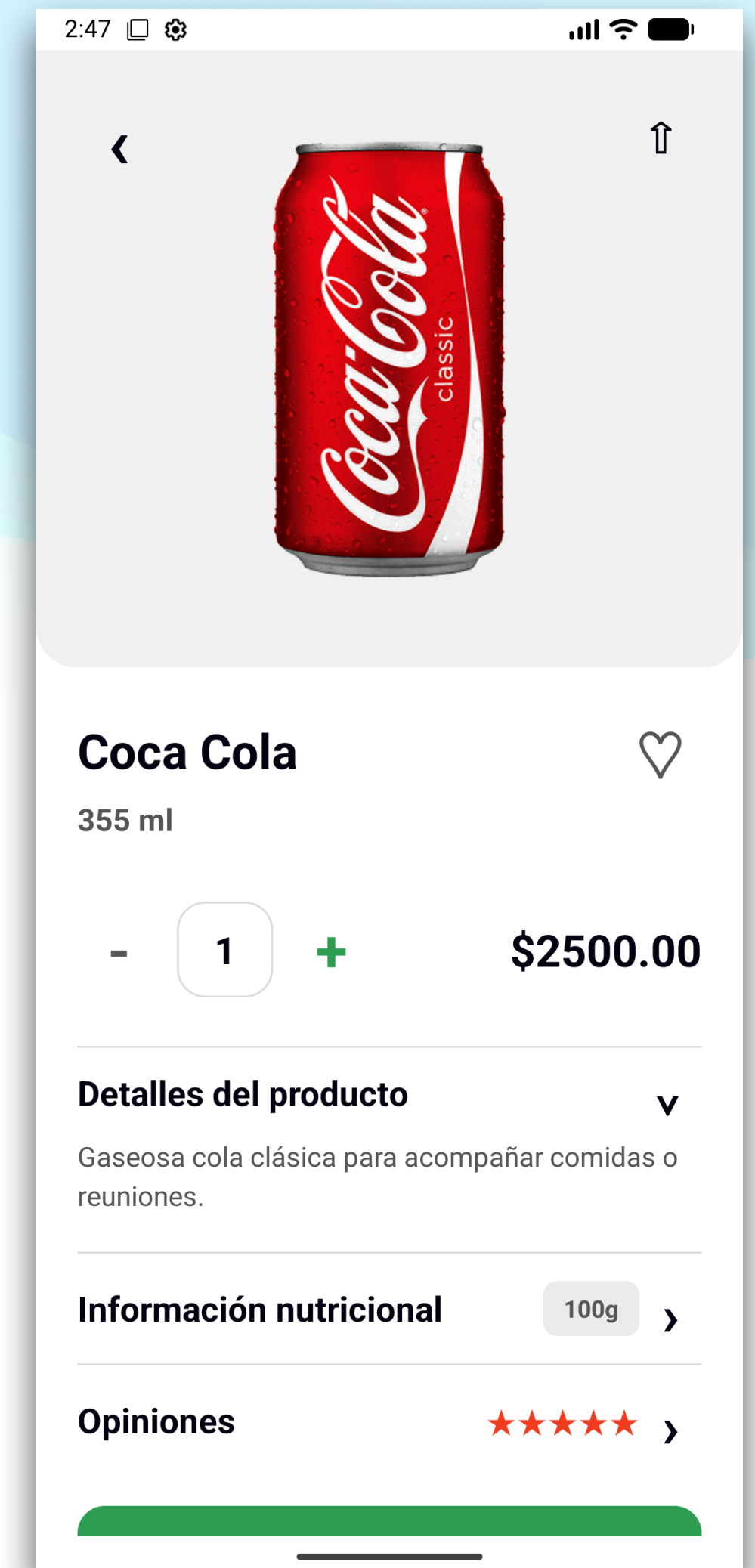
En el detalle se visualiza una imagen grande del producto, su nombre, la presentación, el precio y una breve descripción. También se agregaron secciones visuales de información nutricional y opiniones, siguiendo el diseño de referencia de la aplicación.

Además, esta pantalla incluye controles de cantidad. El usuario puede aumentar o disminuir la cantidad del producto mediante los botones + y -. Al modificar la cantidad, el precio mostrado se actualiza dinámicamente según el valor unitario del producto seleccionado.

También se agregó un botón visual de favorito, que cambia su estado al ser presionado, alternando entre producto favorito y no favorito. Esta acción se informa al usuario mediante mensajes Toast.

El botón **Agregar al carrito** incorpora el producto seleccionado al carrito dinámico, respetando la cantidad elegida por el usuario. Para esto, la pantalla reconstruye un objeto Product con los datos recibidos y lo envía a la clase CartManager, que se encarga de administrar los productos agregados al carrito.

De esta manera, la pantalla de detalle no solo muestra información del producto, sino que también forma parte del flujo principal de compra: selección de producto, modificación de cantidad, agregado al carrito y posterior visualización en la pantalla de carrito.



Pantalla de carrito

La pantalla de carrito fue actualizada para esta entrega final con el objetivo de reemplazar la implementación estática desarrollada en etapas anteriores por una versión dinámica. En esta versión, el contenido del carrito depende de los productos que el usuario agrega desde la pantalla de Bebidas o desde la pantalla de Detalle de producto.

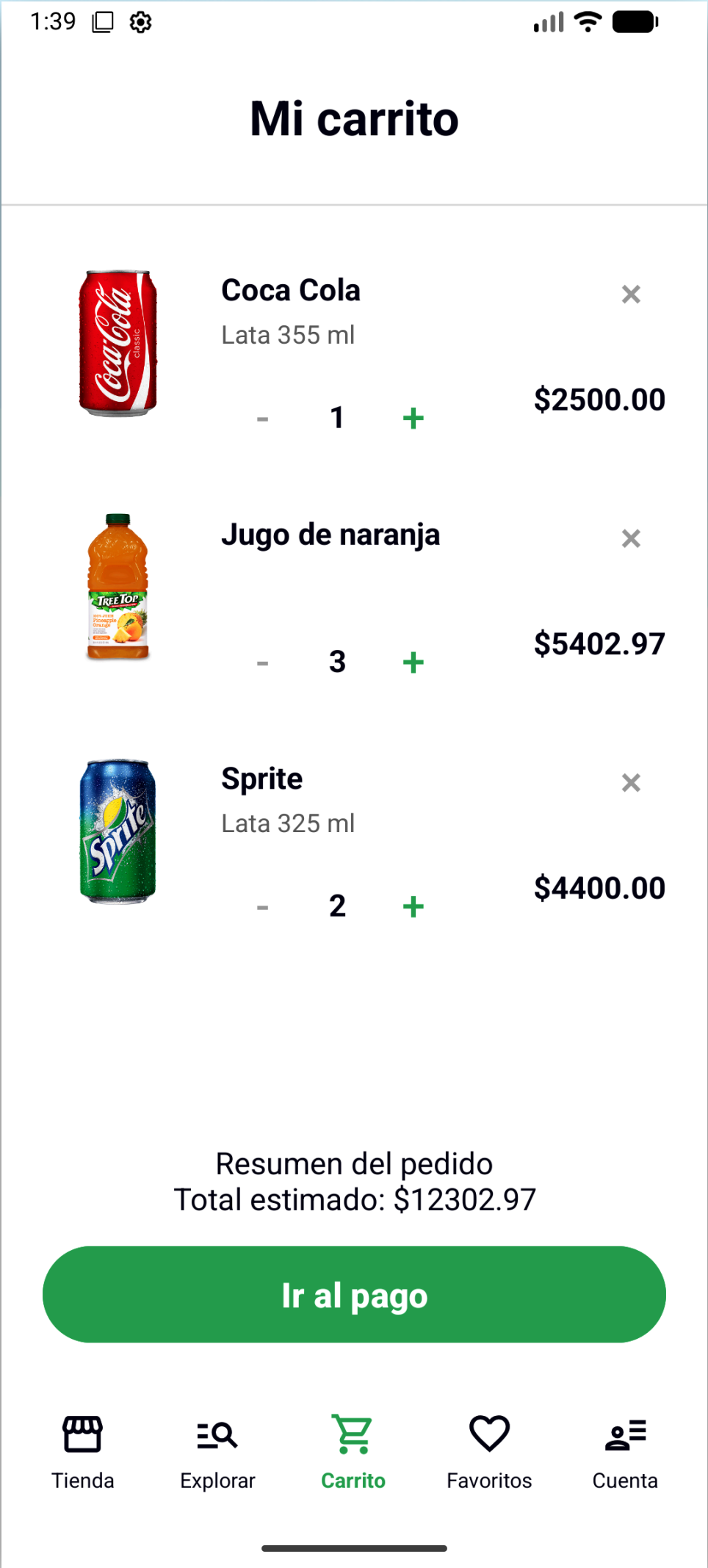
Para implementar esta funcionalidad se incorporaron las clases CartItem, CartManager y CartAdapter. La clase CartItem representa cada producto agregado al carrito junto con su cantidad. La clase CartManager centraliza la lista de productos seleccionados y contiene la lógica para agregar productos, aumentar cantidades, disminuirlas, eliminar elementos y calcular el total. Por último, CartAdapter se encarga de mostrar visualmente los productos dentro de un RecyclerView.

En esta pantalla se muestran los productos agregados por el usuario, junto con su imagen, nombre, presentación, cantidad y precio total según la cantidad seleccionada. Para cada producto se implementaron controles que permiten aumentar o disminuir la cantidad. Al modificar la cantidad, el subtotal del producto se actualiza dinámicamente.

También se agregó la posibilidad de eliminar productos del carrito mediante el botón de cierre ubicado en cada ítem. Cuando un producto es eliminado, se actualiza la lista del carrito. En caso de que todos los productos sean eliminados, la pantalla muestra un mensaje indicando que el carrito se encuentra vacío y deshabilita la acción de continuar con el pago.

El botón **Ir al pago** genera un resumen simple del pedido, calculando el total estimado según los productos agregados y sus cantidades actuales. Esta funcionalidad permite comprobar que el carrito responde a las interacciones realizadas por el usuario durante el recorrido de compra.

Además, la pantalla de carrito se encuentra integrada con la navegación inferior de la aplicación. Desde la barra inferior se puede volver a la pantalla de exploración de productos o ingresar a la pantalla de cuenta del usuario. De esta manera, el carrito forma parte del flujo principal de la app: explorar productos, seleccionar una bebida, revisar el detalle, agregar al carrito y visualizar el pedido.



Pantalla de cuenta

La pantalla de **Cuenta** permite visualizar y administrar información del usuario autenticado en la aplicación. Esta vista se encuentra conectada con Firebase Authentication para identificar al usuario que inició sesión y con Firebase Firestore para consultar y actualizar datos adicionales del perfil.

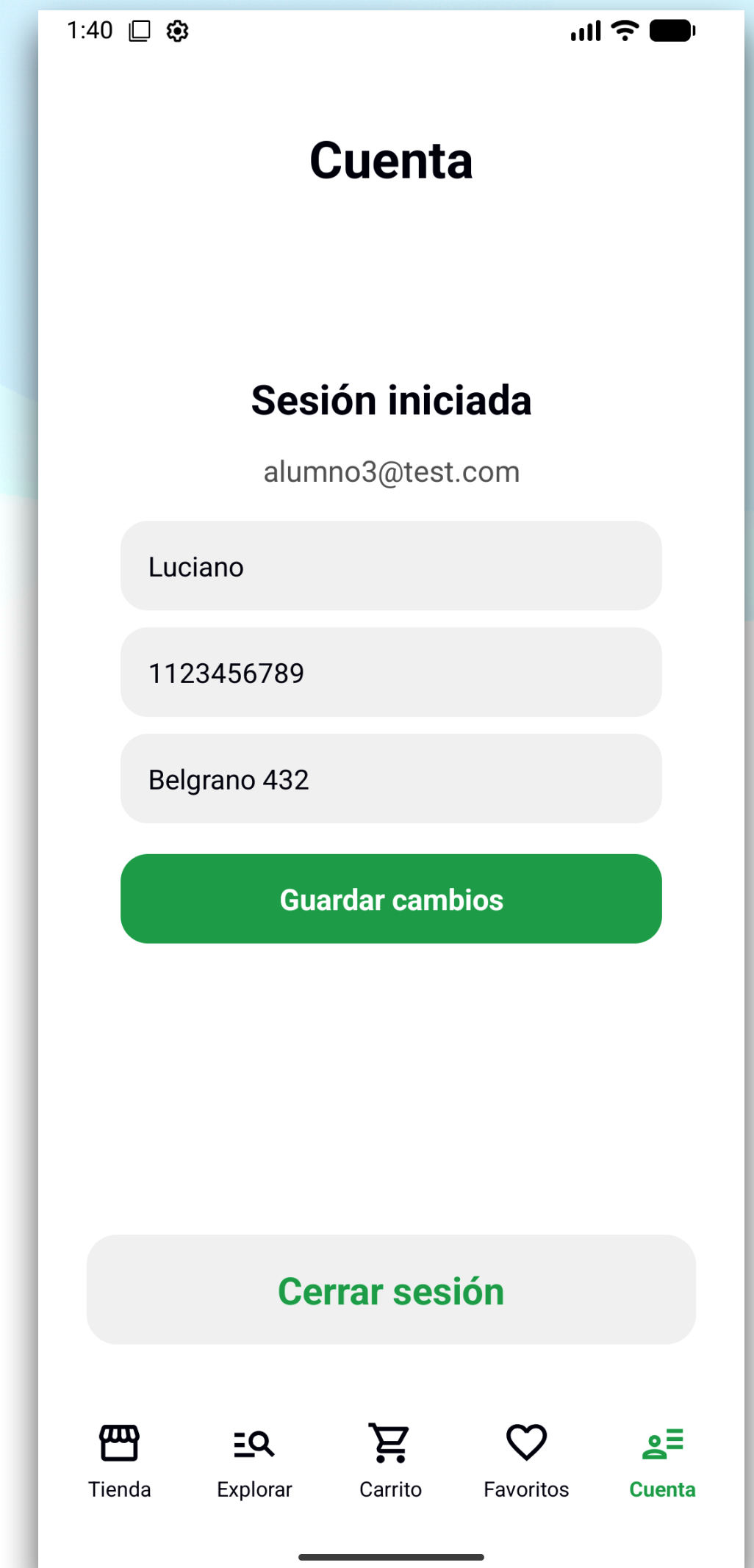
En la parte principal de la pantalla se muestra el correo electrónico correspondiente a la sesión activa. Además, se cargan desde Firestore otros datos asociados al usuario, como nombre, teléfono y dirección. Estos datos se encuentran almacenados en la colección users, dentro de un documento identificado por el uid del usuario autenticado.

La pantalla también permite editar la información del perfil. El usuario puede modificar su nombre, teléfono o dirección y guardar los cambios mediante el botón correspondiente. Al realizar esta acción, la aplicación actualiza el documento del usuario en Firestore y muestra un mensaje informativo indicando el resultado de la operación.

Además, se incluye un botón **Cerrar sesión**. Al presionarlo, la aplicación ejecuta el cierre de sesión mediante Firebase Authentication, elimina la sesión activa del usuario y redirige nuevamente a la pantalla de inicio de sesión.

De esta forma, la pantalla de cuenta completa el flujo de usuario implementado en el proyecto: registro, inicio de sesión, persistencia de sesión, creación de perfil en Firestore, visualización y edición de datos, y cierre de sesión.

También se mantiene la barra de navegación inferior, marcando como activa la sección Cuenta y permitiendo volver a otras secciones principales de la aplicación, como Explorar y Carrito.



Firebase Authentication

Para la autenticación de usuarios se utilizó Firebase Authentication mediante el proveedor de correo electrónico y contraseña. Esta funcionalidad permite registrar usuarios nuevos, validar credenciales existentes, mantener una sesión activa y cerrar sesión desde la aplicación.

En primer lugar, se creó un proyecto en Firebase Console y se registró la aplicación Android utilizando el package name correspondiente al proyecto:

`com.example.parcial1am`

Luego se descargó el archivo `google-services.json` y se incorporó dentro del módulo `app` del proyecto. También se agregaron las dependencias necesarias en los archivos Gradle para poder utilizar los servicios de Firebase dentro de la aplicación Android.

En Firebase Console se habilitó el método de acceso **Correo electrónico/contraseña**, permitiendo que los usuarios puedan registrarse e iniciar sesión mediante esos datos.

Dentro de la aplicación, la lógica de autenticación se implementó principalmente en `LoginActivity`. Para iniciar sesión se utiliza el método `signInWithEmailAndPassword`, que valida el correo y la contraseña ingresados por el usuario. Para registrar un nuevo usuario se utiliza el método `createUserWithEmailAndPassword`.

También se contempló el caso en que un usuario intenta registrarse con un correo que ya existe. En esa situación, la aplicación muestra un mensaje indicando que el correo ya está registrado y que debe iniciar sesión. Además, se agregaron validaciones previas para evitar campos vacíos y contraseñas menores a seis caracteres, ya que Firebase requiere una longitud mínima para las contraseñas.

La aplicación también verifica si ya existe un usuario autenticado al abrir la pantalla de login. Si el usuario ya inició sesión previamente, se lo redirige automáticamente a la pantalla de exploración de productos. Esto permite mantener la sesión iniciada y mejorar la experiencia de uso.

En esta etapa final, Firebase Authentication también se utiliza como base para vincular cada usuario autenticado con su información almacenada en Firestore. Al registrar un nuevo usuario, se obtiene su `uid` y se crea un documento asociado dentro de la colección `users`.

Finalmente, en la pantalla de Cuenta se utiliza Firebase Authentication para obtener el usuario actual y mostrar su correo electrónico. Desde esa misma pantalla se implementó el cierre de sesión mediante `signOut`, redirigiendo nuevamente al usuario hacia la pantalla de inicio de sesión.

Navegación entre pantallas

La aplicación cuenta con un flujo de navegación compuesto por distintas pantallas conectadas entre sí. El recorrido principal comienza en la pantalla de inicio de sesión, donde el usuario puede registrarse o ingresar con una cuenta existente mediante Firebase Authentication.

Una vez autenticado, el usuario accede a la pantalla **Encontrar productos**, que funciona como pantalla principal de exploración. Desde allí puede seleccionar la categoría **Bebidas**, que redirige hacia el listado de productos disponibles para esa categoría.

Desde la pantalla de Bebidas, al tocar una tarjeta de producto, se navega hacia la pantalla de detalle. En esta navegación se envía la información del producto seleccionado mediante Intent y extras, permitiendo que el detalle se complete dinámicamente con los datos correspondientes.

Además, desde la pantalla de Bebidas o desde la pantalla de Detalle, el usuario puede agregar productos al carrito. Estos productos son administrados por la clase CartManager y luego se visualizan en la pantalla de carrito, donde se pueden modificar cantidades, eliminar elementos y calcular el total estimado del pedido.

También se incorporó una barra de navegación inferior que permite moverse entre las secciones principales de la aplicación. Desde esta barra se puede acceder a **Explorar**, **Carrito** y **Cuenta**. Las opciones **Tienda** y **Favoritos** se mantienen visibles como parte del diseño general, mostrando un mensaje informativo al ser seleccionadas.

La pantalla de carrito, desarrollada inicialmente en etapas anteriores, fue actualizada e integrada al flujo general de navegación para mostrar dinámicamente los productos agregados por el usuario. A su vez, la pantalla de Cuenta permite visualizar el usuario autenticado, consultar y editar datos del perfil desde Firestore, y cerrar sesión.

De esta manera, la aplicación deja de estar compuesta por una única pantalla aislada y pasa a tener un recorrido completo entre inicio de sesión, exploración, listado dinámico de productos, detalle, carrito y cuenta de usuario.

Funcionalidades técnicas implementadas

Además del desarrollo visual de las pantallas, se incorporaron distintas funcionalidades técnicas para ampliar el comportamiento de la aplicación y cumplir con los objetivos del trabajo final. En esta etapa se mantuvieron funcionalidades ya desarrolladas, como la autenticación, la navegación entre pantallas y el pasaje de datos mediante Intent, y se agregaron nuevas implementaciones relacionadas con Firestore, productos dinámicos y carrito de compras.

Entre las principales funcionalidades técnicas implementadas se destacan:

Firestore Authentication: se mantiene la autenticación de usuarios mediante correo electrónico y contraseña, permitiendo registro, inicio de sesión, persistencia de sesión y cierre de sesión.

Firestore Firestore: se incorporó Firestore como base de datos remota para guardar, consultar y actualizar información utilizada por la aplicación.

Creación de perfil de usuario: al registrar un nuevo usuario, la aplicación crea automáticamente un documento en la colección users, utilizando el uid del usuario autenticado como identificador.

Lectura y actualización de datos de usuario: desde la pantalla Cuenta se consultan datos del perfil almacenados en Firestore, como nombre, teléfono y dirección. También se permite editar esos datos y guardarlos nuevamente en la base de datos.

Colección de productos: se creó una colección products en Firestore para almacenar los productos de la categoría Bebidas. Cada documento contiene datos como nombre, presentación, precio, descripción, categoría e imagen asociada.

Modelo Product: se incorporó una clase Product para representar los productos obtenidos desde Firestore dentro de la aplicación. Esta clase permite trabajar con los datos de forma más ordenada y reutilizable.

Listado dinámico con RecyclerView: la pantalla de Bebidas fue modificada para mostrar productos mediante un RecyclerView, reemplazando las tarjetas fijas definidas manualmente en el XML.

Adapter de productos: se creó la clase ProductAdapter, encargada de generar visualmente una tarjeta por cada producto obtenido desde Firestore, utilizando un layout reutilizable para cada ítem.

Pasaje de datos con Intent y extras: desde la pantalla de Bebidas se envían los datos del producto seleccionado hacia la pantalla de Detalle, incluyendo identificador, nombre, presentación, precio, descripción, imagen e identificador del recurso visual.

Detalle de producto dinámico: la pantalla de Detalle reutiliza la misma estructura visual para mostrar información distinta según el producto seleccionado por el usuario.

Interacción de cantidad en detalle: se agregaron botones para aumentar y disminuir la cantidad del producto, actualizando el precio mostrado en pantalla según la cantidad seleccionada.

Interacción de favorito: se incorporó un botón de favorito en el detalle de producto, que cambia visualmente al ser seleccionado y muestra mensajes informativos mediante Toast.

Carrito dinámico: se reemplazó la implementación estática del carrito por una versión dinámica, donde los productos agregados desde Bebidas o desde Detalle se visualizan luego en la pantalla de carrito.

Clases para administración del carrito: se incorporaron las clases CartItem, CartManager y CartAdapter. CartItem representa un producto dentro del carrito, CartManager administra la lista de productos agregados y CartAdapter muestra cada elemento dentro del RecyclerView del carrito.

Interacciones del carrito: desde la pantalla de carrito se permite aumentar cantidades, disminuir cantidades, eliminar productos, detectar si el carrito está vacío y calcular el total estimado del pedido.

Buscador de categorías: la pantalla de exploración permite filtrar dinámicamente las categorías según el texto ingresado por el usuario.

Barra de navegación inferior: se mantiene una barra inferior para acceder a las secciones principales de la aplicación, como Explorar, Carrito y Cuenta.

Organización de recursos: se utilizaron archivos de recursos para textos, colores, fondos, íconos e imágenes locales, manteniendo una estructura ordenada dentro del proyecto.

Repositorio del proyecto

Repositorio:

<https://github.com/LucianoGauna/final-am-acn4b-gauna>

Usuario agregado como colaborador:

sergiomedinaio

Rama utilizada:

main

El proyecto amplía la aplicación desarrollada en entregas anteriores, incorporando Firebase Firestore, lectura y actualización de datos remotos, productos dinámicos, uso de RecyclerView, pasaje de datos entre Activities y un carrito de compras dinámico.

Además, se mantiene la autenticación mediante Firebase Authentication, la persistencia de sesión, la navegación inferior entre pantallas principales y la organización de recursos visuales mediante archivos de strings, colors, drawables e imágenes locales.

Durante el desarrollo se utilizó Git y GitHub para versionar los cambios realizados, aplicando mensajes de commit convencionales como feat, fix, refactor, chore y docs.

El nombre del repositorio fue adaptado al formato solicitado para el trabajo final de la materia.